Biomedical Named Entity Recognition Based on Extended Recurrent Neural Networks

Lishuang Li, Liuke Jin, Zhenchao Jiang, Dingxin Song, Degen Huang
School of Computer Science and Technology
Dalian University of Technology
Dalian China
lilishuang314@163.com

***Abstract*—Biomedical named entity recognition (bio-NER), which extracts important entities such as genes and proteins, has become one of the most fundamental tasks in biomedical knowledge acquisition. However, the performance of traditional NER systems is always limited to the construction of complex hand-designed features which are derived from various linguistic analyses and maybe only adapted to specified area. In this paper we mainly focus on building a simple and efficient system for bio-NER with the extended Recurrent Neural Network (RNN) which considers the predicted information from the prior node and external context information (topical information & clustering information). Extracting complex hand-designed features is skipped and replaced with word embeddings. The experiments conducted on the BioCreative II GM data set demonstrate RNN models outperform CRF model and deep neural networks (DNN); furthermore, the extended RNN model performs better than the original RNN model.**

Keywords—bio-NER; recurrent neural network; hand-designed features; word embedding; context information

I. INTRODUCTION

In recent years, with the explosive growth of biomedical information in electronic form, the size of the knowledge repository is rapidly increasing. However, because of the complexity of biomedical nomenclature, biomedical named entity recognition (bio-NER) has been a challenging task. On one hand, it is difficult for biomedical researchers to find information of interest from such a vast database that is continuously updated; on the other hand, inconsistent naming conventions such as "HIV-1 enhancer" versus "HIV 1 enhancer", polysemy of NE (e.g. "TNF alpha" can refer to a protein or DNA) and abbreviations are all increase the difficulty of bio-NER [1]. Therefore, how to build simple and efficient named entity recognition (NER) in biomedical domain is more difficult than that of in open domains.

Though named entity recognition (NER) techniques have typically focused on traditional machine learning methods such as Support Vector Machine (SVM)[2], Maximum Entropy (ME)[3], Hidden Markov Model (HMM)[4] and Conditional Random Fields (CRF)[5], however, their performance and promotion may be affected by some common drawbacks as followings:

1) with the change of corpora and languages, the process to reconstruct feature set is difficult;
2) some complex features with syntactic information rely on the performance of other NLP modules;
3) these features with expert knowledge are expensive to acquire.

In this paper, we mainly apply recurrent neural network (RNN) model to the bio-NER with the hope of tacking/easing the above-mentioned complex hand-designed features issues through the usage of large unlabeled corpus. We extend the Elman-type model [6] through adding reconnection at the output layer and attaching additional feature layer to the raw input and hidden layer. The experimental results show that RNN architecture outperforms deep neural network (DNN) architecture [7], and our extended RNN model further improves the performance of the original RNN model and achieves better results than the shallow models

The rest of this paper is organized as follows: Section 2 introduces our RNN methods for the biomedical named entities recognition. Results and comparison are described in Section 3. Section 4 shows the discussion about our extended model. Finally, conclusions are given in Section 5.

II. METHODLOGY

A. Word embeddings

In our experiments, we use the distributed representations of words trained by Word2Vec tool developed by Mikolov [8], rather than hand-designed features. A real-valued embedding vector is associated with a word, and all the real-valued vectors are obtained by training a skip-gram model from 5.6GB unlabeled text downloaded from PubMed.

And in order to capture short-term temporal dependencies in a sentence, we use a word-context window. With each word mapped to an embedding vector, the word-context window is the ordered concatenation of word embedding vectors. Besides, to process the sentence boundaries, namely the first word and

978-1-4673-6799-8/15/$31.00©2015 IEEE

last word in the sentence, we set a customized vector with same dimension to represent the boundary vectors.

B. Extended recurrent neural network

We extended the Elman-type RNN to adapt it to the bio-NER. The original RNN consists of an input layer, a hidden layer with recurrent connections that propagate time-delayed signals, and an output layer, plus the corresponding weight matrices. In our recognition system, on one hand, output layer with recurrent connection is considered; on the other hand, we increase additional feature layer to model long span context information.

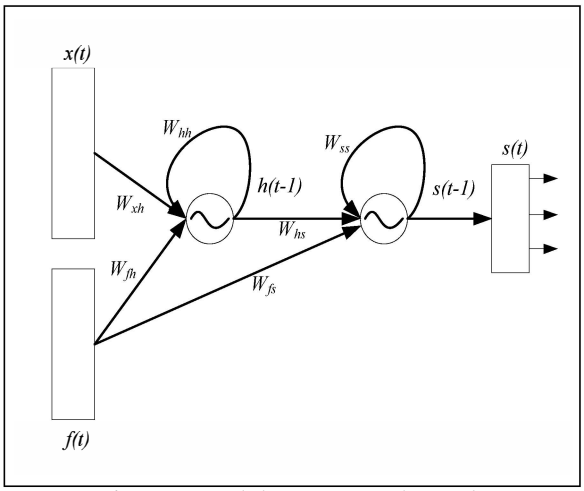

Figure 1: Extended recurrent neural network

1) Extension at the output layer

We extend the Elman-type RNN model by adding recurrent connections at the output layer as shown in Fig. 1. Thus, the hidden layer only needs to be responsible for maintaining the historical information, and output layer will consider the probability information of front node. At each time step, the input is propagated in a standard feed-forward neural network, and then stochastic gradient descent is applied to update the parameters. Through the recurrent connections to time *t-1*, previous information can be propagated to time *t*. And because of this property, the network can maintain and learn a sort of state summarizing past inputs, allowing it to perform sequence prediction. In Fig. 1, hidden layer of the extended RNN can be represented mathematically by (1), and the output layer can be computed as (2).

$$h(t) = sig(x(t) \cdot W_{xh} + h(t-1) \cdot W_{hh} + b_h) \quad (1)$$

$$s(t) = soft(h(t) \cdot W_{hs} + s(t-1) \cdot W_{ss} + b_s) \quad (2)$$

In the above equations, we use (3) as the activation function of the hidden layer.

$$sig(z) = 1/(1+e^{-z}) \quad (3)$$

And at the output layer, we use (4) to predict which tag the current token belongs to.

$$soft(z_m) = e^{z_m} / \sum_k e^{z_k} \quad (4)$$

W_{xh} and W_{hh} are weight matrices between the raw input and the hidden neurons, and between the context hidden neurons and current hidden neurons, respectively. W_{hs} and W_{ss} are weight matrices between hidden layer and output layer, and between previous output node and current output node, respectively. The input vector *x(t)* is formed by concatenating all the word embeddings in the context window of the current word. Hidden layer *h(t-1)* represents output values in the hidden layer from the previous time step, and the output layer *s(t)* produces a probability distribution over labels. Finally, b_h and b_s represent the bias of each layer.

2) Extension with the raw input

Based on the model described above, we give a further extension with an additional feature layer *f(t)* that is connected to both the hidden layers and output layers, as shown in Fig. 1. The feature layer represents an external input vector that should contain complementary information to the input vector *x(t)*. We use two different feature representations such as topic information and clustering information, which can elegantly avoid the long-standing problem of training recurrent neural networks to remember longer-term information to ease the vanishing gradient problem.

- *Topical feature layer.* We take the similar measures with Mikolov and use Latent Dirichlet Allocatioin (LDA) to achieve a compact vector-space representation of long span context [9].
- *Brown clustering feature layer.* Considering clustering algorithm with unlabeled corpus also takes advantage of context information, we constructing an additional class-based input layer using the Brown clustering algorithm [10]. Words are clustered off-line and then the word classes are embedded into the feature layer..

Finally, the output from hidden and output layers can be computed as follows:

$$h(t) = sig(x(t) \cdot W_{xh} + f(t) \cdot W_{fh} + h(t-1) \cdot W_{hh} + b_h) \quad (5)$$

$$s(t) = soft(h(t) \cdot W_{hs} + f(t) \cdot W_{fs} + s(t-1) \cdot W_{ss} + b_s) \quad (6)$$

where W_{fh} and W_{fs} connect the feature layer with hidden layer and output layer respectively.

C. Biomedical named entity recognition using extended RNN

In Fig. 2, we present the process about how to train the ith word in the sentence. Before training, we first randomly initialize two parameters, one is in the hidden layer (i.e. h_0), and the other is in the output layer (i.e. c_0). The two parameters are constantly updated during the training. Then, with raw inputs *x(i-T+1)*, *f(i-T+1)*, h_0 and c_0, we calculate the neurons in the hidden layer and the probabilities in output layer about the first word in the sliding window consisting of *T* words. The outputs of each layer are saved and inputted into the next compute node. We repeat the process till the last word (i.e. i^{th} word) is computed. Finally, the label of ith word can be predicated.

In the Fig. 2, *T* is set to 4 and the process fragment with the dotted box is described in more detail in the Fig. 1. We

consider the hidden layer and output layer $sig_\theta(\cdot)$, $soft_\lambda(\cdot)$ respectively, with parameters θ and λ. The predicated result of ith word in the sentence can be computed with (7) and (8).

$$sig_\theta(\cdot) = sig_\theta^{i}(sig_\theta^{i-1}(\cdots sig_\theta^{i-T+1}\cdots)) \quad (7)$$

$$soft_\lambda(\cdot) = soft_\lambda^{i}(soft_\lambda^{i-1}(\cdots soft_\lambda^{i-T+1}\cdots)) \quad (8)$$

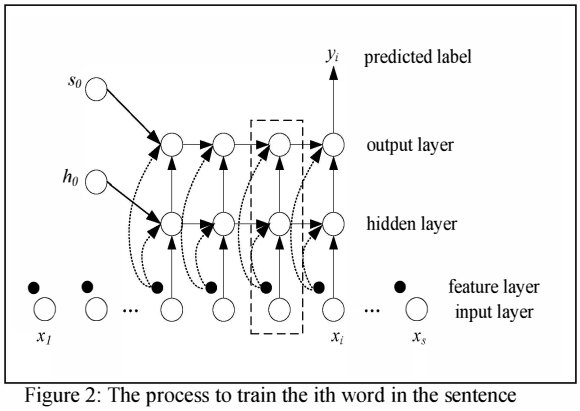

Figure 2: The process to train the ith word in the sentence

III. EXPERIMENTS

Our experiments are carried out on the BioCreative II GM corpus which consists of 15,000 training sentences and 5,000 testing sentences. The results from different models will be shown in this section. We measure the precision, recall and F-score of the improved RNN and compare with CRF model, DNN and original RNNs. In experiments, all the deep networks are based on the common Theano neural network toolkit[1] and all the RNN models are trained with the same hyper-parameters.

A. Results of extended RNN model

TABLE I. RESULT OF EXTENDED RNN WITH DIFFERENT FEATURE LAYERS

Model	Feature layer	P (%)	R (%)	F (%)
Extended RNN	-	82.02	80.12	81.06
	LDA	82.58	79.54	81.03
	LDA(win5)	79.89	82.89	81.36
	Brown	80.93	82.21	81.57
	Brown(win5)	83.29	80.50	**81.87**

In order to explore the impact of richer text information on the extended RNN architecture, we conducted experiments combining different feature layers with the raw word embedding input and hidden layer. In the experiments, the input vector and feature-layer vector are represented with the same dimension. TABLE I reveals that both brown clustering information and topic information have positive effects on the RNN architecture to recognize biomedical named entities. Besides, we also set a feature-context window to represent the current feature vector. The experimental results show that the F-score of our extended RNN model can reach 81.06%. Furthermore, it can rise 0.30% by adding LDA feature layer with 5-word window and 0.81% by adding brown feature layer with the same window.

B. Comparison among different neural networks

TABLE II. RESULTS OF DIFFERENT NEURAL NETWORKS

Model	P (%)	R (%)	F (%)
DNN	74.14	66.86	70.31
Jordan	82.64	79.18	80.87
Elman	82.23	76.40	79.21
Extended RNN +brown(win5)	80.93	82.21	**81.87**

TABLE II lists the results of different neural networks. As the baseline, DNN with one hidden layer can get an F-score of 70.31%, while for RNNs we can improve the performance by 10.56% (Jordan-type RNN [11]) and by 8.90% (Elman-type RNN) respectively compared with DNN. We further extend the two RNN models with an additional brown feature layer, achieving the best performance with 81.87% F-score.

C. Comparison with shallow learning methods

RNN model is easy to build and the raw input vector can be easily obtained by a large-scale unlabeled corpus which is also easily collected from today's explosively increasing knowledge repository. In order to demonstrate that our simple and efficient model has a better performance than the shallow model with common and accessible features, we compare it with the CRF model. And the same word embeddings and basic feature set described as follows are inputted into the CRF model respectively:

Word: Original word and its context.

Stem: We use Genia Tagger for stemming in our method.

Pos: Part-of-speech tag of each word is also obtained by Genia tagger.

Word shape: capitalized characters are replaced by 'A', non-capitalized characters are replaced by 'a', digits are replaced by '0', other characters are unaltered.

Morphological feature 1: Digits in the current word are replaced by a '*' and capital letters are lower cased. If no digits, it is replaced by "no".

Morphological feature 2: each character of the word is replaced by '-', except 5 vowels.

Prefix & Suffix: 2-g, 3-g and 4-g prefix and suffix of the current word.

Orthographic: a total of 29 orthographic features are used in the model.

Word length: the length of the current word which may be 1, 2, 3-5 or >=6.

As shown in TABLE III, when the word embeddings are used as the unique raw input of CRF model, the F-score is 74.38%. When all the accessible hand-designed features listed above are used, the F-score reaches 78.88%. Our extended RNN model attached with the brown-feature layer, adopting

[1] http://deeplearning.net/tutorial/rnnslu.html

the word embeddings, achieves the best F-score of 81.87% which is 7.49% higher than CRF[a], 2.99% higher than CRF[b] .

We also compare our method with Tang et al. [1]'s, which combined some simple hand-designed features such as stemmed words and POS with three types of word representation, and applied them to the CRF model, reaching the best F-score of 80.96%. And the best result of our extended RNN with brown feature layer is higher than that of Tang et al. by 0.91%.

TABLE III. COMPARISON WITH OTHER MACHINE LEARNIG METHOD

Model	P (%)	R (%)	F (%)
CRF[a]	79.79	69.65	74.38
CRF[b]	83.86	74.45	78.88
Tang et al.[1]	86.54	76.05	80.96
Extended RNN+brown(win5)	83.29	80.50	**81.87**

[a.] Only taking the word embedding as the input of CRF model

[b.] *Only taking the basic features listed above as the input of CRF model*

IV. DISCUSSION

From the experimental results, we can conclude that the extended RNN mainly includes three important advantages:

1) *No hand-designed features*. We skip the step of extracting complex hand-designed features, and replace it with word embedding trained off-line. Since high-quality word embeddings can catch a large number of precise syntactic and semantic word relationships [12], this deep learning architecture can fully utilize these information and extract high-level features for the bio-NER.
2) *RNN's property of modeling long span context information.* The results comparing RNN with DNN shows that increased reconnection in the deep neural network can have an obvious improvement for the recognition. This mainly because that the recurrent property of RNN can maintain a long span historical information and learn what to store and what to ignore.
3) *Additional reconnection and feature layer*. Considering that predicated result (i.e. probability of labeling) from prior node can have an important impact on the current prediction, we extend the original Elman-type RNN model by adding a reconnection at the output layer. Besides, the added external input vector connected to both hidden and output layers should contain complementary information to the raw input. And this vector can be used to convey contextual information about the sentence being modeled. With this information, the extended model will remember longer-term information and ease the inherent issue in RNN (the vanishing gradient problem). From the experimental results, we can see that both extended methods can produce positive impact on the bio-NER.

V. CONCLUSION

In this paper, we mainly focus on establishing a simple, rapid and efficient system for biomedical named entity recognition (bio-NER). We skip the step which extracts various complex artificial features and replace it with word embeddings trained by unlabeled corpus. Then we apply the extended RNN model which adds additional reconnection at the output layer to capture probability information and feature layers to rich raw input to the bio-NER task. The experimental results show that our extended model can achieve better performance.

Acknowledgment

The authors gratefully acknowledge the financial support provided by the National Natural Science Foundation of China under No. 61173101, 61173100, and the State Key Science & Technology Project for Infectious Diseases (2012ZX10002011). The Titan X used for this research was donated by the NVIDIA Corporation.

References

[1] B. Tang, H. Cao, X. Wang, Q. Chen, and H. Xu, "Evaluating Word Representation Features in Biomedical Named Entity Recognition Tasks," BioMed research international 2014.

[2] K. J. Lee, Y. S. Hwang, S. Kim, and H. C. Rim, "Biomedical named entity recognition using two-phase model based on SVMs.," Journal of Biomedical Informatics 37.6 , 2004, pp. 436-447.

[3] S. K. Saha, S. Sarkar, and P. Mitra, "Feature selection techniques for maximum entropy based biomedical named entity recognition.," Journal of biomedical informatics 42.5, 2009, pp. 905-911.

[4] D. Shen and C. Tan, "Effective Adaptation of a Hidden Markov Model-based Named Entity Recognizer for Biomedical Domain," In Proceedings of the ACL 2003 workshop on Natural language processing in biomedicine-Volume 13 , pp. 49-56, July 2003.

[5] C. Sun, Y. Guan, X. Wang, and L. Lin, "Rich features based Conditional Random Fields for biological named entities recognition.," Computers in Biology and Medicine37.9, 2007, pp. 1327-1333.

[6] J. L. Elman, "Finding Structure in Time," Cognitive Science, 14(2), 1990, pp. 179-211.

[7] J. Weston and M. Karlen, "Natural Language Processing (Almost) from Scratch," The Journal of Machine Learning Research, 2011, pp. 2493-2537.

[8] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient Estimation of Word Representations in Vector Space," arXiv preprint arXiv:1301.3781, January. 2013.

[9] D. M. Blei, A. Y. Ng, and M. I. Jordan, "Latent Dirichlet Allocation," the Journal of machine Learning research, 3, 2003, pp. 993-1022.

[10] P. F. Brown, P. V. Desouza, R. L. Mercer, V. J. D. Pietra, and J. C. Lai. "Class-based n-gram models of natural language," Computational linguistics, 18(4), 1992, pp. 467-479.

[11] M. I. Jordan. "Serial order: A parallel distributed processing approach," Advances in psychology, 121, 1997, pp. 471-495 .

[12] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Distributed Representations of Words and Phrases and their Compositionality," In Advances in neural information processing systems, 2013, pp. 3111-3119.